# TP4

## Système RAG avec génération et évaluation par un modèle de langage

---

### 1. Introduction

Les modèles de langage (LLM) permettent de générer des réponses en langage naturel à partir de questions formulées par les utilisateurs. Cependant, ils peuvent produire des informations incorrectes ou non justifiées par les données disponibles. Pour limiter ce problème, l'approche RAG (*Retrieval-Augmented Generation*) associe la recherche d'informations dans une base documentaire à la génération d'une réponse par un modèle de langage.

Ce travail pratique consiste à concevoir un système RAG intégrant une base vectorielle, un modèle de génération et un modèle chargé d'évaluer les réponses produites. Le système est développé en Python dans un notebook, à l'aide de bibliothèques open source.

Les expériences utilisent une base de connaissances fictive correspondant au Campus Atlas. Cette base sert uniquement à illustrer le fonctionnement du système et ne représente pas un établissement réel.

### 2. Objectifs du TP

Les principaux objectifs de ce travail sont les suivants :

— Construire une base de connaissances à partir de documents textuels.
— Découper les documents en segments adaptés à la recherche sémantique.
— Transformer les textes en représentations vectorielles grâce aux embeddings.
— Retrouver les passages les plus pertinents pour une question.
— Générer une réponse à partir du contexte récupéré.
— Évaluer la réponse à l'aide d'un modèle de langage distinct dans son rôle et son prompt.
— Tester le système avec une question connue, une question sans réponse disponible et une réponse volontairement incorrecte.

### 3. Environnement et technologies utilisées

Le notebook a été exécuté dans un environnement Python sur Google Colab, avec un fonctionnement sur processeur (CPU). Les bibliothèques utilisées permettent de préparer les documents, de construire l'index vectoriel et d'exécuter les modèles localement.

— **LangChain :** organisation des documents et coordination des composants du système.
— **Sentence Transformers :** génération des embeddings multilingues à partir du modèle `paraphrase-multiling`
— **ChromaDB :** stockage et recherche des vecteurs.
— **Transformers et PyTorch :** chargement et exécution du modèle génératif.
— **Qwen2.5-0.5B-Instruct :** modèle local utilisé pour la génération et l'évaluation.

L'objectif est de réaliser les expériences sans clé API ni crédit OpenAI. Les modèles nécessaires sont téléchargés lors de la première exécution, ce qui nécessite une connexion Internet. L'exécution sur CPU reste possible, mais elle peut être plus lente qu'avec un GPU.

## 4. Préparation de la base documentaire

La première étape consiste à créer une base de connaissances composée de six documents textuels. Ces documents décrivent des informations fictives relatives au Campus Atlas, notamment les horaires de la bibliothèque, les ateliers d'intelligence artificielle et les services disponibles.

Aucun fichier PDF n'est utilisé : les informations sont directement définies sous forme de texte. Chaque document est associé à une source permettant d'identifier son origine lors de la recherche.

Cette organisation constitue la base de référence à partir de laquelle le système doit retrouver les informations pertinentes. La qualité des réponses dépend donc en partie de la précision et de la pertinence des documents fournis.

### 4.1 Découpage en segments

Les documents sont ensuite découpés en segments, appelés *chunks*, à l'aide de `RecursiveCharacterTextSplitter`. Les paramètres retenus sont une taille maximale de 300 tokens et un chevauchement de 20 tokens entre segments consécutifs.

Le chevauchement permet de conserver une partie du contexte entre deux segments lorsque les documents sont plus longs. Dans notre expérience, les documents étant courts, le résultat obtenu est de six chunks pour six documents.

## 5. Indexation et recherche sémantique

Après le découpage, chaque segment est transformé en vecteur numérique par le modèle d'embeddings multilingue. Ces vecteurs représentent les caractéristiques sémantiques des textes et permettent de rechercher des passages proches du sens d'une question.

Les vecteurs et les documents associés sont indexés dans ChromaDB. Le composant *VectorDB* utilise ensuite une recherche par similarité pour retrouver les trois segments les plus pertinents pour une question donnée ($k = 3$).

Lors du test portant sur les horaires de la bibliothèque le samedi, la recherche retourne trois passages. Le premier concerne directement le règlement de la bibliothèque ; les autres passages proviennent d'autres documents de la base.

Cette étape fournit au modèle génératif un contexte documentaire susceptible de répondre à la question, sans lui transmettre nécessairement l'ensemble de la base de connaissances.

## 6. Génération et évaluation des réponses

### 6.1 Composant LLM1

Le composant *LLM1* est responsable de la génération de la réponse. Il reçoit la question de l'utilisateur et le contexte extrait par la recherche vectorielle. Son prompt lui demande de répondre brièvement

en français et de s'appuyer uniquement sur les informations fournies.

L'utilisation du contexte vise à limiter les réponses non justifiées. Cependant, cette consigne ne garantit pas à elle seule l'absence d'erreurs, en particulier avec un modèle de petite taille.

### 6.2 Composant JudgeLLM

Le composant *JudgeLLM* intervient après la génération. Il examine la question, le contexte documentaire et la réponse produite, puis tente d'attribuer une note comprise entre 1 et 5, accompagnée d'une justification.

Son objectif est d'estimer si la réponse est cohérente avec les informations disponibles. Il joue ainsi le rôle d'un évaluateur automatique. Le format de sortie attendu est structuré, mais le modèle peut ne pas toujours le respecter.

Il est important de distinguer cette évaluation automatique d'une validation humaine : la note produite constitue une estimation et peut elle-même être incorrecte.

### 6.3 Orchestration par le composant RAG

Le composant *RAG* coordonne les différentes étapes du système. Son fonctionnement suit l'ordre suivant :

1. Recevoir la question de l'utilisateur.
2. Rechercher les passages pertinents avec *VectorDB*.
3. Construire le contexte à partir des passages récupérés.
4. Générer une réponse avec *LLM1*.
5. Transmettre la question, le contexte et la réponse à *JudgeLLM*.
6. Retourner la réponse et les informations d'évaluation.

Le même contexte est conservé pour la génération et l'évaluation. Cette organisation facilite la comparaison entre les informations retrouvées et le contenu de la réponse.

## 7. Expériences et résultats

Trois tests ont été réalisés afin d'étudier le comportement du système.

### 7.1 Test 1 : question avec réponse disponible

La première question concerne les horaires de la bibliothèque du Campus Atlas le samedi.

**Résultat obtenu :** le système génère la réponse indiquant une ouverture de 9 h à 13 h. Cette information correspond au document de référence consacré au règlement de la bibliothèque.

Le juge attribue une note de 5/5 et indique que la réponse correspond aux informations du contexte. Dans ce cas, la réponse et son évaluation sont cohérentes avec les données disponibles.

### 7.2 Test 2 : question sans réponse dans la base

Le deuxième test demande le nom du directeur du Campus Atlas. Or, aucun document de la base ne fournit cette information.

Le système produit néanmoins le nom « Jean-Paul », qui n'est pas justifié par les documents. Le juge attribue pourtant une note de 5/5, avec une justification qui ne permet pas de confirmer la véracité de cette information.

Ce résultat met en évidence une limite importante : le modèle génératif peut inventer une information absente du contexte, tandis que le modèle juge peut ne pas détecter cette erreur. Le comportement attendu aurait été de signaler que l'information n'est pas disponible.

### 7.3 Test 3 : réponse volontairement incorrecte

Dans le troisième test, une réponse erronée est soumise directement au juge. Elle affirme que la bibliothèque est ouverte le samedi de 20 h à minuit, alors que le document de référence indique une ouverture de 9 h à 13 h.

Le juge attribue malgré tout une note de 5/5 et considère la réponse comme complète et concrète. Cette évaluation est incorrecte, puisque les horaires annoncés contredisent les données documentaires.

Ce test montre que le juge utilisé ne garantit pas la détection des contradictions et des informations inventées. Il est donc nécessaire d'améliorer son prompt, de vérifier les critères de notation et de réaliser des évaluations supplémentaires.

## 8. Discussion et limites

Ce travail pratique permet de mettre en place les principaux composants d'un système RAG : préparation des documents, génération des embeddings, indexation vectorielle, recherche sémantique, génération de réponses et évaluation automatique.

Les résultats montrent que le système peut fournir une réponse correcte lorsque l'information recherchée est présente dans la base. Cependant, les deux derniers tests révèlent des limites importantes. Le modèle génératif ne respecte pas toujours les informations disponibles, et le juge peut attribuer une note élevée à une réponse incorrecte.

Plusieurs améliorations sont envisageables :

— Renforcer les consignes demandant de reconnaître l'absence d'information dans le contexte.
— Améliorer le prompt du juge afin qu'il vérifie explicitement chaque affirmation par rapport aux sources.
— Utiliser un modèle d'évaluation plus performant et comparer ses résultats avec des évaluations humaines.
— Ajouter des tests sur plusieurs questions, notamment des réponses correctes, incomplètes, contradictoires et non justifiées.
— Examiner les passages récupérés pour vérifier leur pertinence avant la génération de la réponse.

Enfin, les expériences reposent sur un corpus fictif limité à six documents et sur un modèle de petite taille exécuté sur CPU. Les résultats ne suffisent donc pas à conclure que le système serait fiable dans un environnement réel.

## 9. Conclusion

Ce TP a permis de réaliser un système RAG combinant une base documentaire textuelle, des embeddings multilingues, ChromaDB et un modèle de langage local. L'architecture développée sépare la recherche de l'information, la génération de la réponse et son évaluation automatique.

Le premier test a montré que le système pouvait répondre correctement à une question lorsque l'information était présente dans les documents. En revanche, les tests suivants ont révélé que le modèle pouvait inventer des informations et que le juge ne détectait pas toujours les réponses incorrectes.

Ainsi, le travail réalisé constitue une base fonctionnelle pour comprendre l'architecture RAG, mais des améliorations restent nécessaires pour renforcer la fidélité aux sources et la fiabilité de l'évaluation.